

Object Oriented Programming, often referred to as OOP, is a common programming paradigm found in many high level languages such as javascript and Python. In OOP we will make and manipulate Objects through the use of Classes to handle *data* and *logic*. Let's run through the very basics of OOP.

Classes

A Class is a blueprint to create an object. When we create an object from a class we call this new object an "instance" of the Class. Typically, we will need to store this object in a variable so that we can access it later. All classes contain a list of properties and methods(), so any instance of a class will have these properties and methods as well. In code, we will always see Classes as beginning with a capital letter.

A Class's properties are used to store *data* about the object. We can think of these as adjectives. Note that properties will always be written in all lower case.

A Class's methods() are used to perform *logic* related to the object. We can think of these as verbs. Note that methods() are written in all lower case but have a set of parentheses on the end. These parentheses are used to pass *arguments* to the method, essentially extra instructions for what the object will do. Some methods will take *multiple arguments*, when multiple arguments are needed we separate them with a ",". Not all methods() will require extra arguments, but many will.

Inheritance

In OOP classes will typically have *inheritance*. This is where a Class will be built from a *parent* Class, *inheriting* all of its properties and methods(). Consider the following example.

Our *parent* Class in this example is the Pet Class. Our Pet Class has the following properties and methods():

Pet

properties	description	methods()	description
age	The pet's age in years	feed()	Gives the pet food
weight	The pet's weight in kg	play()	Plays with the pet
size	The pet's size in cubic meters		
name	The pet's name		

This Pet class is very general but gives information that every pet would have. Some times we will have a *child* Class that will add more specificity. Here's a *child* Class for Pet: Dog.

Dog

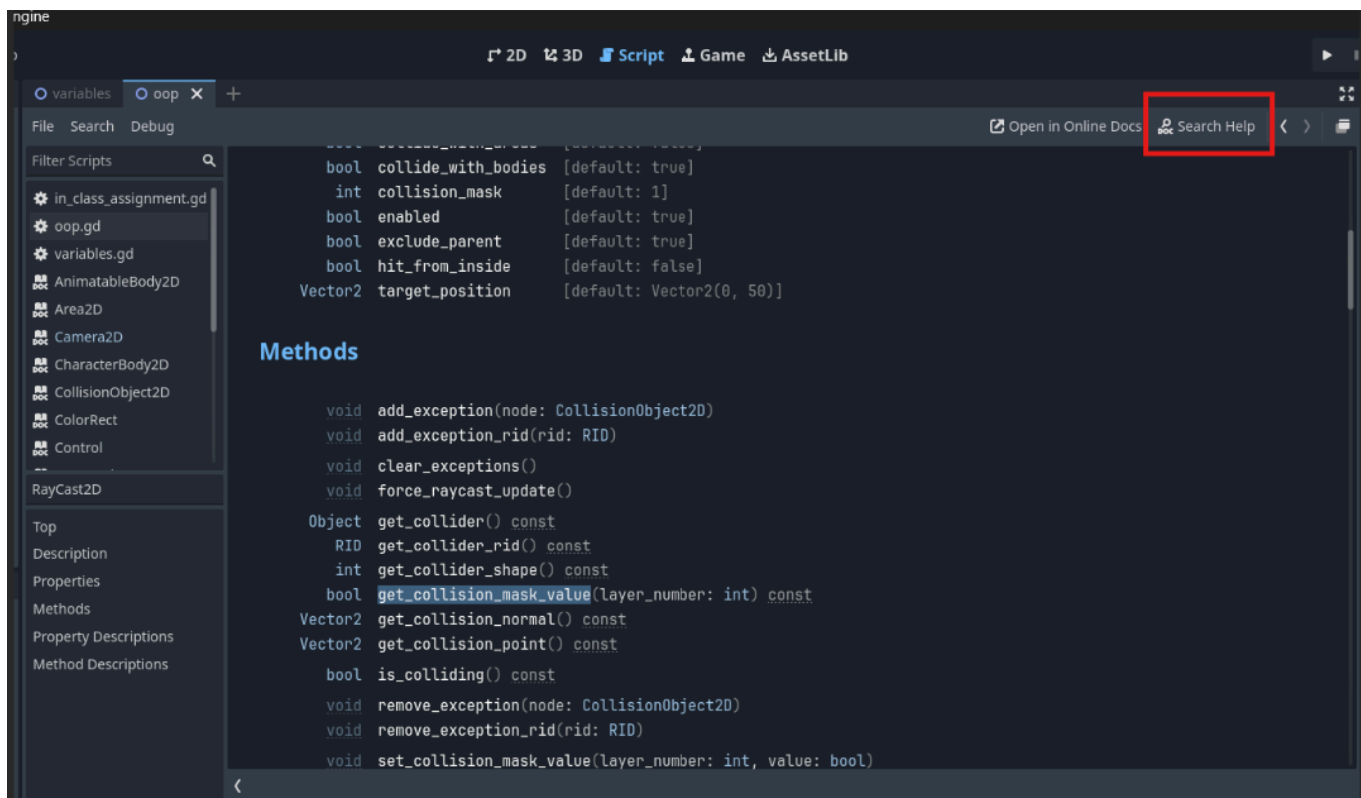
properties	description	methods()	description
breed	The breed of the dog	jump()	The dog jumps
legs	The amount of legs the dog has	fetch()	Tells the dog to retrieve another Object
tail_length	How long the dog's tail is in meters	sit()	The dog sits
collar	If the dog has a collar or not (bool)	pet()	Pets the dog
		walk()	Takes the dog for a walk

Often, a *child* Class will not display the properties and methods() of its *parent* Class in its documentation. Remember though, our Dog Class inherits *all* of Pet's properties and methods(). Meaning that we would be able to use the Dog's name property or its feed() method.

Finding a Node's properties and methods() in Godot

In GDScript we will refer to Classes as Nodes (they're the same thing!). We can find a Node's properties and methods() by looking at its *documentation*. Godot has its documentation built into the engine, which makes finding information out about a Node fairly easy. We can access a Node's documentation in two easy ways:

The "search help" button



The "search help" button can be found at the top right of the viewport when in the Script view. This button allows you to search through all of the documentation by either typing the name of a Node or searching through the Node list.

Ctrl+Click

We may also ctrl+click on any Node name (or property or method(!)) in a script to automatically pull up its documentation. This can be very helpful if we forget how a Node or method() works and need to quickly see its documentation.

Now it's your turn!

1. For the following Node, list all of its properties and methods() (please list them separately, not mixed together) found on its documentation page. Then list all of the properties and methods() found on its first *parent* Class's documentation page (the properties and methods() it inherits):

Sprite2D

2. For the following properties of the Control node, list their data types they contain:

rotation

size

auto_translate

focus_next

shortcut_context

3. For the following methods() of RayCast2D list how many arguments it takes, what data types they are, if it returns a value, and if it does return a value the kind of data that is returned.

set_collision_mask_value()

get_collider()

get_collision_mask_value()

4. For the following code, identify if the line is a property, a method, or a Node (Class)

```
Node2D
position
look_at()
Vector2
is_colliding()
```